

Pianificazione e ottimizzazione di traiettorie per la robotica collaborativa

26 agosto 2026

Sommario

Il presente lavoro di tesi si colloca nell'ambito della robotica collaborativa (Human-Robot Collaboration, HRC) e presenta lo sviluppo di un'architettura software di controllo allo scopo di garantire la sicurezza durante le attività lavorative. L'obiettivo del percorso è l'implementazione in C++ di un algoritmo di arresto ottimo, basato sull'ottimizzazione del tempo di arresto e sul controllo d'impedenza. Questo approccio viene in seguito confrontato con una metodologia basata su forze repulsive e sul controllo d'ammettenza.

Per gestire le zone di sicurezza, il manipolatore robotico e l'operatore umano vengono modellati come una serie di capsule geometriche (Sphere-Swept Lines, SSL) con dimensioni dinamiche, variabili in base allo stato cinematico (velocità e accelerazioni) del robot. Questa modellazione semplifica notevolmente il calcolo delle distanze relative, garantendo la gestione delle interazioni in tempo reale.

La validazione sperimentale è stata condotta su un cobot Franka Emika Panda a 7 gradi di libertà. Il tracciamento dell'operatore nello spazio tridimensionale è affidato a una telecamera di profondità Intel RealSense D435 accoppiata all'algoritmo YOLOv8 per la stima della posa. Lo scambio di dati a bassa latenza tra il nodo di visione e il nodo di controllo è realizzato tramite la libreria di messaggistica ZeroMQ.

Infine, le due strategie di controllo vengono confrontate sulla base di metriche prestazionali concrete: tempo totale di esecuzione (T_{TASK}), tempo di inattività del robot (R_{IDLE}), tempo di attività concorrente nello spazio di lavoro (C_{ACT-WS}) e numero di arresti del robot (R_{STOPS}).

Indice

Sommario	i
Elenco delle figure	v
1 Introduzione	1
1.1 Collaborazione uomo-robot	1
1.2 Sicurezza nella collaborazione uomo-robot	3
1.3 Obiettivo della tesi	5
1.4 Struttura della tesi	5
2 Approcci considerati	7
2.1 Rappresentazione geometrica del sistema	7
2.1.1 Sphere Swept Lines (SSL)	7
2.1.2 Modellazione del robot e dell'operatore	8
2.2 Distanza geometrica tra robot e operatore	10
2.3 Primo approccio: dimensionamento dinamico delle zone di sicurezza	11
2.3.1 Principio dello Speed and Separation Monitoring (SSM)	12
2.3.2 Pianificazione online della traiettoria di arresto	12
2.3.3 Formulazione dell'ottimizzazione e vincoli dinamici . .	14
2.4 Secondo approccio: capsule <i>soft</i>	14
2.4.1 Confronto tra capsule <i>hard</i> e <i>soft</i>	15
2.4.2 Controllo di ammettenza per l'evasione dinamica . . .	15
3 Implementazione	19
3.1 Braccio robotico	19
3.2 Architettura software e strumenti di modellazione	20
3.2.1 Controllo del robot e libreria <i>libfranka</i>	20
3.2.2 Gestione della dinamica: <i>Pinocchio</i>	22
3.2.3 Modellazione attriti ai giunti	24
3.2.4 Ambiente di simulazione	26

3.3	Sistema di visione e playback dello scheletro	27
3.4	Primo metodo: frenata morbida	27
3.4.1	Ottimizzazione tramite CasADi	27
3.4.2	Rilevamento di una potenziale collisione	30
3.4.3	Esecuzione del Task	30
3.5	Secondo approccio: capsule <i>soft</i>	36
4	Risultati	39
4.1	BOZZA	39
4.2	Controllo di posizione e di coppia ai giunti	39
4.3	Primo approccio	39
4.4	Secondo approccio	39
5	Conclusioni e sviluppi futuri	41
	Bibliografia	43

Elenco delle figure

1.1	Rappresentazione schematica di una cella di lavoro collaborativa. Viene evidenziata la sovrapposizione tra il volume operativo dell'operatore (<i>Human work envelope</i>) e l'area di lavoro del robot (<i>Robot work envelope</i>).	2
1.2	Rappresentazione schematica del principio di Speed and Separation Monitoring (SSM).	4
2.1	Rappresentazione geometrica di una Sphere Swept Line (SSL).	8
2.2	Confronto tra la geometria dettagliata del robot (a) e la sua rappresentazione semplificata a capsule (b).	9
2.3	Rappresentazione geometrica della distanza minima d_{seg} tra due segmenti S_1 e S_2	11
2.4	Profili temporali di posizione $q(t)$, velocità $\dot{q}(t)$ e accelerazione $\ddot{q}(t)$ durante la manovra di arresto morbido con polinomio quintico.	13
3.1	Robot collaborativo <i>Franka Emika Panda</i>	19
3.2	Confronto tra le coppie reali misurate ai giunti e le coppie stimate tramite il modello dinamico in Pinocchio.	25
3.3	Architettura software del sistema di controllo. L'alto livello gestisce la pianificazione offline e la ri-pianificazione delle traiettorie, mentre il basso livello esegue il loop di controllo ad alta frequenza (1 kHz) alternando deterministicamente gli stati di RUNNING e STOPPING	33

Capitolo 1

Introduzione

1.1 Collaborazione uomo-robot

La robotica industriale ha assunto un ruolo sempre più rilevante nei moderni processi produttivi, grazie alla possibilità di automatizzare attività ripetitive, incrementare la produttività e garantire elevati livelli di precisione e ripetibilità. Tradizionalmente, l'impiego di robot industriali è stato caratterizzato dalla separazione fisica tra l'area di lavoro dell'operatore e quella del robot, mediante l'utilizzo di barriere, recinzioni o altri dispositivi volti a impedire l'accesso dell'uomo alla zona operativa durante il movimento del robot. Questo approccio permette di ridurre il rischio di interazione accidentale, separando nettamente le attività svolte dall'uomo e quelle affidate al sistema robotico. Tale separazione, tuttavia, può risultare poco conveniente in applicazioni nelle quali le capacità del lavoratore e quelle del robot possono essere integrate per ottenere un processo più efficiente e flessibile.

Da questa esigenza nasce il concetto di collaborazione uomo-robot, nel quale l'operatore umano e la macchina robotica operano all'interno di uno spazio condiviso, come schematizzato in Figura 1.1, e possono contribuire simultaneamente alla realizzazione di uno stesso compito [1]. In questo contesto, il robot non viene più considerato come un sistema autonomo e separato dall'operatore, ma come uno strumento in grado di interagire con esso e di supportarne le attività. La condivisione dello spazio di lavoro permette quindi di combinare le caratteristiche complementari dell'uomo e del robot, con l'obiettivo di ottenere sistemi produttivi più flessibili e adattabili rispetto alle soluzioni di automazione tradizionali.

Le applicazioni della collaborazione uomo-robot sono principalmente legate a situazioni nelle quali le capacità dell'operatore e quelle del robot risultano complementari [2]. Il robot può, ad esempio, occuparsi di attività caratteriz-

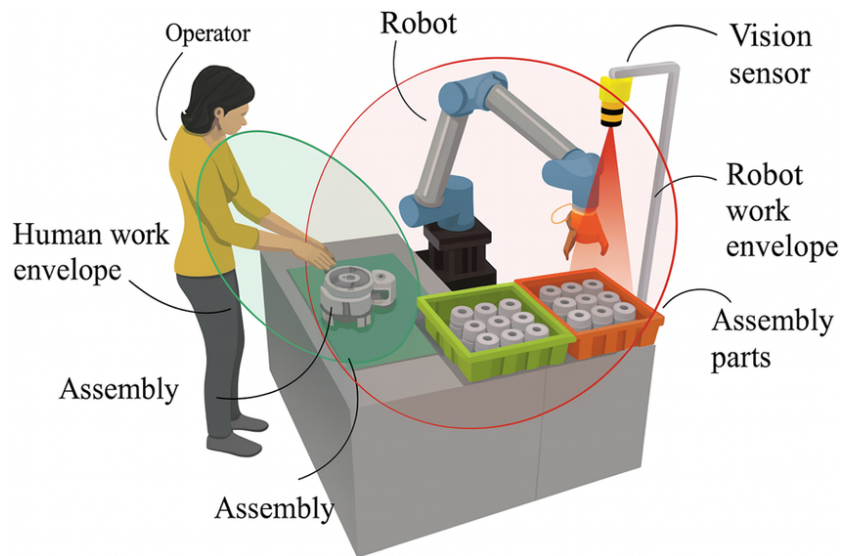


Figura 1.1: Rappresentazione schematica di una cella di lavoro collaborativa. Viene evidenziata la sovrapposizione tra il volume operativo dell'operatore (*Human work envelope*) e l'area di lavoro del robot (*Robot work envelope*).

zate da elevata ripetibilità, precisione o movimentazione di carichi, mentre l'operatore può intervenire nelle operazioni che richiedono capacità decisionali, adattamento a condizioni variabili o manipolazioni più complesse. Questa suddivisione delle attività permette di sfruttare i vantaggi dell'automazione mantenendo, allo stesso tempo, la flessibilità e la capacità di adattamento proprie dell'operatore umano.

La collaborazione diretta tra uomo e robot può quindi trovare applicazione in diversi processi industriali, tra cui assemblaggio, movimentazione, manipolazione e attività di supporto all'operatore. In questi scenari, la possibilità di lavorare all'interno dello stesso spazio consente di ridurre la necessità di separare fisicamente le due attività e di progettare processi nei quali uomo e robot possano intervenire in maniera complementare sullo stesso task.

La condivisione dello spazio di lavoro introduce tuttavia una problematica fondamentale: la necessità di garantire condizioni di sicurezza durante l'interazione. A differenza delle applicazioni tradizionali, nelle quali la presenza dell'operatore nella zona di movimento del robot viene impedita, nei sistemi collaborativi la presenza umana deve essere considerata come una condizione normale di funzionamento. La gestione della sicurezza assume quindi un ruolo centrale nella progettazione di questi sistemi e rappresenta uno degli aspetti fondamentali per consentire una collaborazione efficace tra uomo e robot.

1.2 Sicurezza nella collaborazione uomo-robot

La possibilità di condividere lo spazio di lavoro tra operatore umano e robot introduce la necessità di adottare specifiche strategie volte a garantire la sicurezza dell'interazione [3]. Il rischio associato alla collaborazione dipende infatti da diversi fattori, tra cui la geometria del robot, la velocità di movimento, le caratteristiche dell'ambiente di lavoro e la posizione dell'operatore rispetto al sistema robotico.

Un primo elemento che può contribuire alla riduzione delle conseguenze di un eventuale contatto è rappresentato dalla progettazione del robot. La geometria e i materiali utilizzati per la realizzazione dei componenti che possono entrare in contatto con l'operatore possono infatti influenzare le conseguenze di una collisione. In particolare, l'impiego di superfici arrotondate e l'assenza di spigoli vivi permettono di ridurre la possibilità che un eventuale contatto provochi lesioni all'operatore. Sebbene questi accorgimenti contribuiscano alla sicurezza del sistema, essi non sono sufficienti a garantire autonomamente condizioni di sicurezza durante l'interazione.

La sicurezza dei sistemi robotici collaborativi è pertanto regolamentata da specifici standard internazionali, che definiscono principi e requisiti per la progettazione e la valutazione delle applicazioni di collaborazione uomo-robot [4]. Tra gli aspetti considerati rientrano anche le condizioni che devono essere rispettate affinché l'operatore possa condividere lo spazio di lavoro con il robot in condizioni di sicurezza.

Un aspetto particolarmente rilevante è rappresentato dalla distanza tra l'operatore e il robot. Durante il movimento, infatti, è necessario che il sistema sia in grado di rilevare una situazione potenzialmente pericolosa con sufficiente anticipo da permettere l'adozione di un'azione correttiva prima che si verifichi un contatto. La distanza necessaria non può quindi essere considerata come un valore fisso, ma deve essere valutata tenendo conto della dinamica del sistema e, in particolare, della velocità del robot e dell'operatore e dei tempi necessari affinché il sistema possa reagire alla situazione di rischio [5].

Un approccio alla gestione di questo problema è rappresentato dallo *Speed and Separation Monitoring* (SSM), nel quale la distanza tra l'operatore e il robot viene monitorata durante l'esecuzione del task. Con riferimento alla Figura 1.2, quando la separazione tra i due è tale da non garantire più le condizioni di sicurezza, il sistema può intervenire modificando il comportamento del robot, ad esempio riducendone la velocità (zona gialla) o arrestandone il movimento (zona rossa). La distanza necessaria per garantire la sicurezza dipende quindi dalle condizioni di funzionamento del sistema e deve esse-

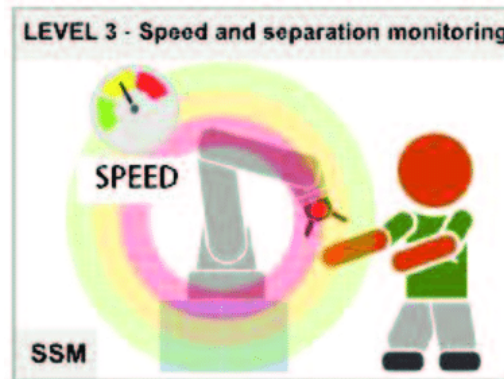


Figura 1.2: Rappresentazione schematica del principio di Speed and Separation Monitoring (SSM).

re valutata considerando anche il tempo necessario affinché il robot possa effettuare l'azione prevista.

Nell'ambito dello SSM sono state sviluppate diverse strategie finalizzate a migliorare il compromesso tra sicurezza e continuità operativa. Un arresto del robot ogni volta che l'operatore si avvicina alla zona di lavoro permette di adottare un comportamento conservativo, ma può comportare frequenti interruzioni dell'attività e conseguenti riduzioni della produttività. Per questo motivo, sono state studiate strategie che permettono di adattare dinamicamente le condizioni di sicurezza in funzione del movimento del robot e dell'operatore. In particolare, Scalera et al. [6] propongono una strategia basata sullo scaling online delle zone di sicurezza dinamiche, con l'obiettivo di migliorare la fluidità e la produttività della collaborazione mantenendo le condizioni di sicurezza.

Il presente lavoro si inserisce in questo contesto e si concentra sul confronto tra due differenti strategie software per la gestione di una possibile situazione di collisione. Le due procedure utilizzano le informazioni relative alla posizione dell'operatore e al movimento del robot per valutare le condizioni di sicurezza, ma adottano strategie differenti per la gestione del movimento. La prima procedura prevede l'arresto del robot quando viene prevista una possibile collisione, mentre la seconda modifica il movimento del robot con l'obiettivo di mantenere una distanza di sicurezza dall'operatore senza ricorrere necessariamente all'arresto completo.

1.3 Obiettivo della tesi

L'obiettivo del presente lavoro di tesi è quello di sviluppare e analizzare due strategie di controllo per la collaborazione sicura uomo-robot, entrambe basate sull'utilizzo della visione artificiale per il rilevamento della presenza e della posizione dell'operatore all'interno dello spazio di lavoro del robot.

La prima strategia, arresto forzato, deriva dagli studi di Scalera et al. [6], che hanno analizzato l'impiego di capsule di ingombro dinamiche, chiamate *Sphere Swept Lines* o SSL, secondo il paradigma di *Speed and Separation Monitoring* o SSM. In particolare viene calcolato il tempo minimo di arresto del robot in funzioni di vari vincoli del robot. La seconda strategia estende il primo approccio introducendo una tecnica di modifica del movimento del robot basato su un controllo di ammettenza [7]. In questo modo il robot mantiene una distanza di sicurezza modificando la propria traiettoria, evitando se possibile l'arresto completo del robot. Il confronto viene effettuato attraverso le seguenti metriche:

- durata complessiva di esecuzione del task
- tempo durante il quale il robot è fermo
- numero di arresti
- tempo durante il quale operatore e robot risultano contemporaneamente attivi all'interno dello spazio di lavoro

Lo studio mira quindi a determinare le caratteristiche e le prestazioni delle due strategie e ad evidenziarne vantaggi e limiti in relazione all'impiego in scenari di collaborazione uomo-robot.

1.4 Struttura della tesi

La tesi è organizzata in cinque capitoli che descrivono il processo di riconoscimento del problema, implementazione di una soluzione e confronto tra i due approcci software.

Il primo capitolo introduce il contesto della collaborazione uomo-robot, descrivendo le principali problematiche legate alla sicurezza dell'interazione e presentando gli obiettivi del lavoro.

Nel secondo capitolo vengono descritti i due approcci proposti per la gestione della sicurezza. Per ciascuna procedura vengono illustrati il principio di funzionamento, le grandezze considerate e la strategia adottata per la gestione del movimento del robot.

Il terzo capitolo è dedicato all'implementazione sperimentale. Vengono presentati il sistema robotico utilizzato, l'ambiente di sviluppo, l'implementazione delle due procedure e la configurazione degli esperimenti. Vengono inoltre definite le metriche utilizzate per il confronto e il protocollo sperimentale adottato.

Nel quarto capitolo vengono presentati e discussi i risultati ottenuti durante la fase sperimentale. Le due procedure vengono confrontate sulla base delle metriche definite, analizzandone il comportamento in termini di sicurezza, continuità operativa ed efficienza nell'esecuzione del task.

Il quinto capitolo, infine, riassume i principali risultati del lavoro e presenta le conclusioni dello studio, evidenziando vantaggi e limiti delle due strategie utilizzate e indicando possibili sviluppi futuri.

Capitolo 2

Approcci considerati

2.1 Rappresentazione geometrica del sistema

Prima di definire le strategie di gestione della sicurezza, è necessario introdurre una rappresentazione geometrica del robot e dell'operatore che possa essere impiegata per effettuare le verifiche di prossimità in tempo reale. La geometria reale di un manipolatore e del corpo umano è infatti complessa e una sua modellazione dettagliata risulterebbe computazionalmente troppo onerosa per i calcoli da eseguire continuamente durante il movimento. Per questo motivo, entrambi vengono approssimati mediante volumi geometrici semplificati (primitive geometriche).

2.1.1 Sphere Swept Lines (SSL)

Una delle rappresentazioni più diffuse per questo scopo è costituita dalle *Sphere Swept Lines* (SSL), le quali permettono di approssimare la geometria di un corpo tramite l'unione di un segmento e di una sfera. Nel caso di un segmento di estremi P_0 e P_1 , la SSL corrispondente è definita formalmente come la somma di Minkowski tra il segmento $\overline{P_0P_1}$ e una sfera di raggio r [8].

In generale, dati due insiemi A e B nello spazio euclideo, la loro somma di Minkowski è definita come:

$$A + B = \{a + b \mid a \in A, b \in B\}. \quad (2.1)$$

Nel caso specifico delle SSL, la somma di Minkowski tra il segmento e la sfera genera un solido a forma di capsula, interpretabile come il luogo dei punti nello spazio aventi distanza minore o uguale a r dal segmento $\overline{P_0P_1}$.

L'impiego delle SSL consente quindi di sostituire una geometrie tridimensionali articolate con un insieme contenuto di primitive, semplificando

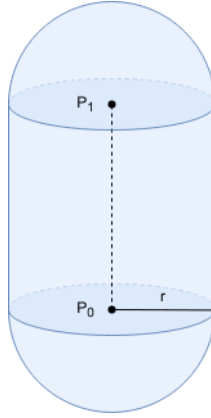


Figura 2.1: Rappresentazione geometrica di una Sphere Swept Line (SSL).

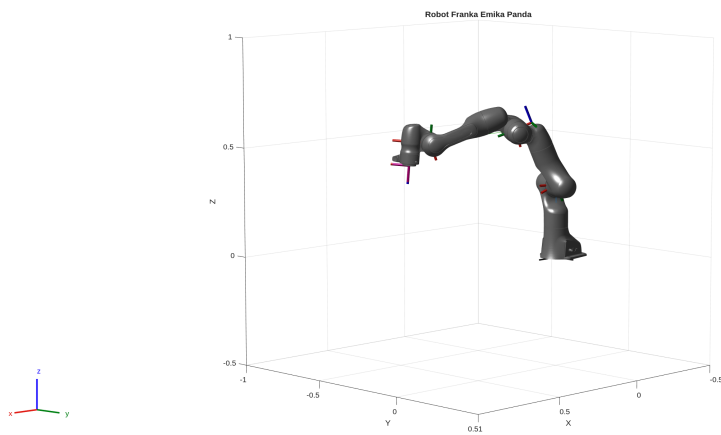
notevolmente il calcolo algebrico delle distanze minime e l'individuazione di eventuali intersezioni tra robot e operatore.

2.1.2 Modellazione del robot e dell'operatore

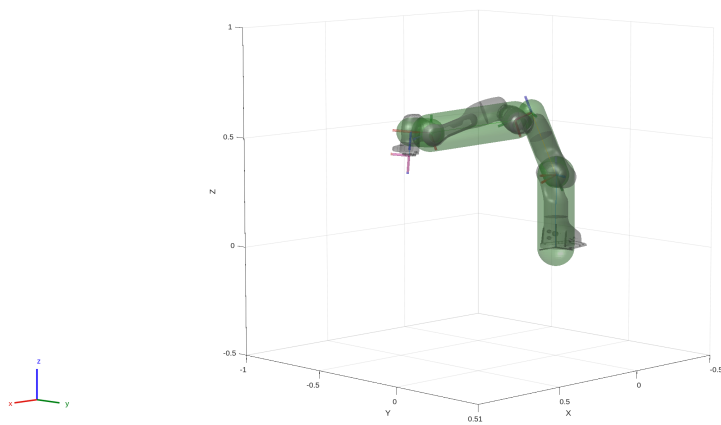
Nel presente lavoro, il robot utilizzato per le prove sperimentali è un manipolatore Franka Emika Panda a sette gradi di libertà. Ai fini della valutazione della distanza dall'operatore, la struttura del robot viene approssimata tramite un insieme di quattro SSL (capsule), associate rispettivamente alle coppe di giunti principali: base-1, 2-3, 4-5, 6-flangia. In figura 2.2 si può osservare

I segmenti delle capsule vengono definiti in tempo reale a partire dalla cinematica diretta del robot, quindi dalle posizioni dei giunti, mentre il raggio r di ciascuna SSL viene opportunamente dimensionato in modo da racchiudere interamente il volume del braccio robotico nel tratto corrispondente. L'aggiornamento continuo della posizione dei giunti durante il moto permette di ricalcolare istante per istante la posa delle SSL, consentendo quindi di effettuare le verifiche di sicurezza in tempo reale.

In modo analogo, l'operatore umano viene modellato attraverso una scomposizione geometrica basata sullo scheletro tridimensionale rilevato dal sistema di visione. Tramite opportuni algoritmi di *skeleton tracking* viene fornito un insieme ordinato di punti 3D corrispondenti alle principali articolazioni anatomiche (testa, spalle, gomiti, polsi, anche). Tali punti vengono utilizzati come estremi dei segmenti su cui vengono definite le SSL associate alle diverse parti del corpo dell'operatore.



(a) Robot Panda nella configurazione iniziale



(b) Modellazione geometrica tramite SSL

Figura 2.2: Confronto tra la geometria dettagliata del robot (a) e la sua rappresentazione semplificata a capsule (b).

2.2 Distanza geometrica tra robot e operatore

Come introdotto nella sezione precedente, sia il robot che l'operatore umano vengono approssimati tramite insiemi di primitive geometriche a capsula (SSL). Di conseguenza, la determinazione della distanza minima di sicurezza si riconduce al problema geometrico del calcolo della distanza minima tra i segmenti nello spazio tridimensionale $\mathbb{R}^3$, a cui viene successivamente sottratta la somma dei rispettivi raggi.

Si considerino due segmenti S_1 e S_2 , individuati dalle rispettive coppie di estremi (P_1, Q_1) e (P_2, Q_2) . Qualsiasi punto appartenente alle rette su cui giacciono tali segmenti può essere espresso in forma parametrica come:

$$L_1(s) = P_1 + sD_1, \quad \text{con } D_1 = Q_1 - P_1 \quad (2.2)$$

$$L_2(t) = P_2 + tD_2, \quad \text{con } D_2 = Q_2 - P_2 \quad (2.3)$$

dove $s, t \in \mathbb{R}$ sono i parametri di linea. I punti appartengono ai segmenti fisici S_1 e S_2 se e solo se i rispettivi parametri soddisfano i vincoli $s \in [0, 1]$ e $t \in [0, 1]$.

Definito il vettore di connessione tra le origini dei due segmenti $R = P_1 - P_2$, il vettore che unisce due punti generici sulle due rette è espresso da:

$$v(s, t) = L_1(s) - L_2(t) = R + sD_1 - tD_2. \quad (2.4)$$

Il problema di minima distanza equivale a minimizzare la norma al quadrato $\|v(s, t)\|^2$. Impostando le condizioni di ortogonalità del segmento di minima distanza rispetto alle direzioni dei due segmenti, ovvero $v \cdot D_1 = 0$ e $v \cdot D_2 = 0$, si ottiene un sistema di equazioni lineari il cui annullamento fornisce i valori ottimi dei parametri per le rette non parallele:

$$s = \frac{(D_1 \cdot D_2)(D_2 \cdot R) - (D_2 \cdot D_2)(D_1 \cdot R)}{(D_1 \cdot D_1)(D_2 \cdot D_2) - (D_1 \cdot D_2)^2} \quad (2.5)$$

$$t = \frac{(D_1 \cdot D_1)(D_2 \cdot R) - (D_1 \cdot D_2)(D_1 \cdot R)}{(D_1 \cdot D_1)(D_2 \cdot D_2) - (D_1 \cdot D_2)^2} \quad (2.6)$$

Qualora i valori di s e t così calcolati rientrino entrambi nell'intervallo $[0, 1]$, i punti di minima distanza C_1 e C_2 si trovano all'interno dei segmenti e sono definiti da:

$$C_1 = L_1(s) = P_1 + sD_1 \quad (2.7)$$

$$C_2 = L_2(t) = P_2 + tD_2 \quad (2.8)$$

Nel caso in cui uno o entrambi i parametri giacciono all'esterno dell'intervallo $[0, 1]$, i punti di minima distanza si trovano sui bordi di almeno uno

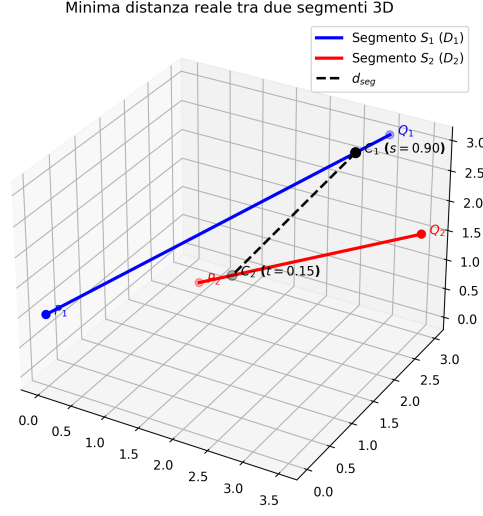


Figura 2.3: Rappresentazione geometrica della distanza minima d_{seg} tra due segmenti S_1 e S_2 .

dei due segmenti. In tale eventualità, si satura il parametro fuori limite all'estremo più vicino, cioè 0 o 1, e si ricalcola analiticamente il valore dell'altro parametro mediante proiezione ortogonale sul relativo segmento, garantendo così il rispetto dei vincoli geometrici.

Come mostrato in Figura 2.3, trovati i due punti di minima distanza C_1 e C_2 , la distanza euclidea tra i segmenti centrali risulta:

$$d_{seg} = \|C_1 - C_2\| \quad (2.9)$$

In conclusione, la distanza effettiva d_{SSL} tra le due primitive geometriche a capsula, caratterizzate dai rispettivi raggi r_1 e r_2 , è ricavata sottraendo i due raggi dei volumi di ingombro:

$$d_{SSL} = \max(0, d_{textseg} - r_1 - r_2) \quad (2.10)$$

2.3 Primo approccio: dimensionamento dinamico delle zone di sicurezza

Questo approccio alla collaborazione uomo-robot in sicurezza si basa sul principio di *Speed and Separation Monitoring* (SSM). Per superare la rigidità dei metodi convenzionali con zone di sicurezza fisse o sovradimensionate, il sistema ottimizza online il tempo di arresto T_{stop} .

2.3.1 Principio dello Speed and Separation Monitoring (SSM)

Secondo le direttive della specifica tecnica ISO/TS 15066 [4], la modalità di collaborazione SSM impone che la velocità operativa del manipolatore sia continuamente regolata in funzione della distanza relativa dall'operatore umano. L'obiettivo principale è garantire che la distanza di separazione corrente $S(t)$ tra qualsiasi punto del robot e l'operatore non sia mai inferiore a una distanza di sicurezza minima $S_{\min}(t)$, definita come:

$$S_{\min}(t) = S_h(t) + S_r(t) + S_s(t) + C + Z_d + Z_r \quad (2.11)$$

dove:

- $S_h(t)$ rappresenta lo spostamento potenziale dell'operatore verso il robot durante la fase di reazione ed esecuzione dell'arresto;
- $S_r(t)$ è lo spazio percorso dal robot durante il tempo di reazione del sistema di controllo t_r ;
- $S_s(t)$ rappresenta lo spazio effettivo di frenata del manipolatore;
- C , Z_d e Z_r sono termini cautelativi legati rispettivamente alle capacità di intrusione dell'operatore, alle incertezze dei sensori di tracciamento e alla tolleranza di posizionamento del robot.

Nei sistemi tradizionali, le componenti $S_s(t)$ e $S_r(t)$ vengono calcolate assumendo le peggiori condizioni cinematiche e dinamiche possibili (es. massima velocità e carico nominale), conducendo a distanze $S_{\min}$ costanti, conservative e fortemente penalizzanti per la produttività. Per superare tale rigidità, il metodo qui descritto propone un'ottimizzazione in tempo reale del tempo di arresto T_{stop} , consentendo la generazione di capsule protettive dinamiche e adeguate all'effettivo stato cinematico del robot. La stima accurata di T_{stop} consente di determinare l'effettivo spazio di frenata e di ridimensionare dinamicamente le capsule di sicurezza protettive, modellate come *Sphere Swept Lines* (SSL), attorno ai bracci del manipolatore.

2.3.2 Pianificazione online della traiettoria di arresto

Al momento della rilevazione di un potenziale rischio di collisione al generico istante temporale t_0 , il controllore non applica un'interruzione discontinua della traiettoria nominale. Viene invece generata online una traiettoria di arresto continua e differenziabile del dominio dei giunti per evitare sollecitazioni impulsive sui riduttori.

Si definisce t_r il tempo di reazione dell'architettura di controllo. la manovra di frenata si sviluppa nell'intervallo temporale $t \in [t_i, t_f]$ dove $t_i = t_0 + t_r$ rappresenta l'istante di innesco reale della frenata e $t_f = t_0 + t_r + T_{\text{stop}}$ specifica l'istante di arresto completo [9].

Per imporre la continuità di posizione, velocità e accelerazione, la traiettoria di arresto $q_j(t)$, per $j = 1, \dots, 7$, viene descritta tramite un polinomio del 5° grado [9], per ciascuno dei 7 giunti del manipolatore, :

$$q_j(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5 \quad (2.12)$$

I coefficienti a_k , per $k = 0, \dots, 5$, del polinomio vengono determinati analiticamente in tempo reale imponendo le condizioni al contorno tra lo stato cinematico iniziale di frenata al tempo t_i e lo stato di arresto completo al tempo t_f :

$$\begin{cases} q_j(t_i) = q_{i,j}, & q_j(t_f) = q_{i,j} \\ \dot{q}_j(t_i) = \dot{q}_{i,j}, & \dot{q}_j(t_f) = 0 \\ \ddot{q}_j(t_i) = \ddot{q}_{i,j}, & \ddot{q}_j(t_f) = 0 \end{cases} \quad (2.13)$$

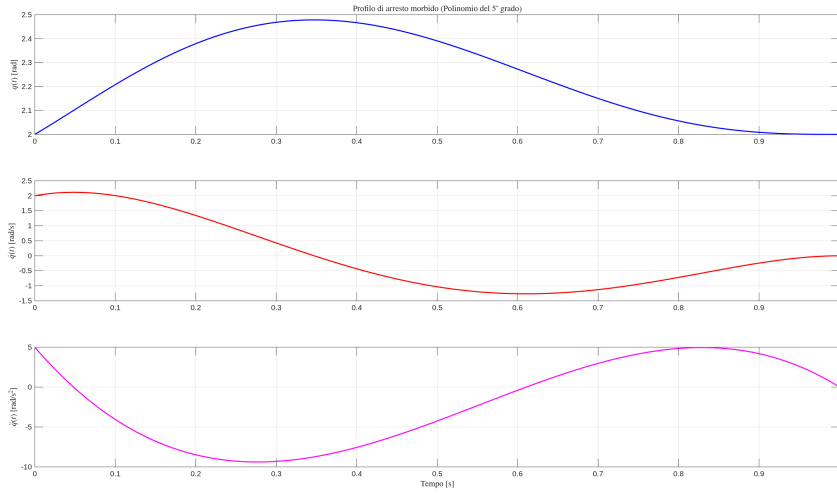


Figura 2.4: Profili temporali di posizione $q(t)$, velocità $\dot{q}(t)$ e accelerazione $\ddot{q}(t)$ durante la manovra di arresto morbido con polinomio quintico.

Si noti che in questa formulazione si impone che la posizione obiettivo di arresto al tempo t_f corrisponda alla posizione $q_{i,j}$ registrata all'istante di innesco della frenata, come si può notare dalla Figura 2.4. Sarà compito della pianificazione polinomiale quintica garantire la continuità e la sicurezza dell'arresto lungo l'intervallo T_{stop} , gestendo la transizione a partire dalla velocità iniziale non nulla $\dot{q}_{i,j} \neq 0$.

2.3.3 Formulazione dell'ottimizzazione e vincoli dinamici

La determinazione del tempo di arresto minimo T_{stop} viene formulata come un problema di ottimizzazione vincolata ad ogni passo di controllo [9, 6]:

$$\min_{T_{\text{stop}}} w_0 T_{\text{stop}} + w_1 |T_{\text{stop}} - T_{\text{stop,prev}}| \quad (2.14)$$

dove $w_0, w_1 > 0$ sono pesi di ponderazione e $T_{\text{stop,prev}}$ rappresenta la soluzione computata al ciclo precedente. La presenza del termine penalizzante $|T_{\text{stop}} - T_{\text{stop,prev}}|$ introduce un'azione di regolarizzazione, garantendo continuità temporale nelle dimensioni delle capsule ed evitando variazioni brusche dell'area di ingombro tra iterazioni successive.

L'ottimizzazione dell'equazione 2.14 è soggetta al rispetto stringente dei limiti cinematici e dinamici del manipolatore per tutta la durata della frenata, $\forall t \in [t_i, t_f]$:

$$|\dot{q}_j(t)| \leq \dot{q}_{j,\max} \quad \forall j = 1, \dots, 7 \quad (2.15)$$

$$|\ddot{q}_j(t)| \leq \ddot{q}_{j,\max} \quad \forall j = 1, \dots, 7 \quad (2.16)$$

$$|\dddot{q}_j(t)| \leq \dddot{q}_{j,\max} \quad \forall j = 1, \dots, 7 \quad (2.17)$$

$$|\tau_j(t)| \leq \tau_{j,\max} \quad \forall j = 1, \dots, 7 \quad (2.18)$$

$$|\dot{\tau}_j(t)| \leq \dot{\tau}_{j,\max} \quad \forall j = 1, \dots, 7 \quad (2.19)$$

Il vettore delle coppie ai giunti $\tau(t) \in \mathbb{R}^n$ è legato allo stato cinematico attraverso il modello dinamico inverso dell'equazione di Lagrange:

$$\tau(t) = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) \quad (2.20)$$

In cui $M(q) \in \mathbb{R}^{7 \times 7}$ rappresenta la matrice d'inerzia, $C(q, \dot{q}) \in \mathbb{R}^{7 \times 7}$ include i termini centripeti e di Coriolis e $g(q) \in \mathbb{R}^7$ è il vettore delle forze di gravità.

Attraverso la soluzione dell'equazione (2.14), il sistema calcola il tempo di frenata minimo ammissibile. Ciò consente di modellare capsule di sicurezza protettive con l'ingombro spaziale strettamente necessario, massimizzando lo spazio di lavoro condiviso ed evitando arresti non necessari durante l'esecuzione del task.

2.4 Secondo approccio: capsule *soft*

Per minimizzare le interruzioni operative e garantire una continuità di servizio nello spazio di lavoro collaborativo, il secondo approccio introduce l'impiego di capsule *soft*. Rispetto ai volumi di ingombro impiegati per la gestione

degli arresti di sicurezza, queste capsule presentano dimensioni in generale maggiori e definiscono una fascia di interazione dinamica tra il robot e l'operatore umano. In questo scenario, un'eventuale intersezione tra le capsule del manipolatore e le *Sphere Swept Lines* (SSL) associate all'operatore non determina un arresto immediato del sistema, ma innesca una strategia di evasione attiva basata sul ricalcolo in tempo reale della traiettoria del task[7].

2.4.1 Confronto tra capsule *hard* e *soft*

Per armonizzare i requisiti di produttività e sicurezza, questa architettura prevede l'adozione simultanea di due set concentrici di capsule:

- **Capsule *hard*:** presentano dimensioni prossime al volume d'ingombro fisico del manipolatore e svolgono una funzione puramente protettiva. Come descritto nella Sezione 2.3, una loro eventuale intersezione con le SSL dell'operatore indica una violazione dei margini minimi di sicurezza e comporta l'attivazione immediata dell'arresto con traiettoria di frenata morbida a profilo quintico.
- **Capsule *soft*:** possiedono un raggio maggiorato e sono vincolate alla struttura dei bracci robotici. Esse definiscono una zona di pre-allarme dinamica: la misura del grado di penetrazione o sovrapposizione con le SSL dell'operatore non arresta il robot, ma viene impiegata per sintesi e calcolo della forza repulsiva virtuale responsabile della deviazione di traiettoria.

L'azione combinata dei due set garantisce che il robot cerchi prima di evitare attivamente l'operatore tramite le capsule *soft* e, solo nel caso in cui l'evasione non sia sufficiente a prevenire la violazione delle capsule *hard*, intervenga la procedura di arresto sicuro.

2.4.2 Controllo di ammettenza per l'evasione dinamica

Il ricalcolo della traiettoria in tempo reale si fonda sull'implementazione di uno schema di controllo di ammettenza nello spazio cartesiano. Nell'ambito del controllo dell'interazione, la letteratura distingue tra approcci d'impedenza e d'ammettenza in base alle relazioni di causalità adottate:

- **Controllo d'impedenza (Causalità Moto $\rightarrow$ Forza):** accetta un errore/deviazione di posizione come ingresso e calcola la forza o coppia di reazione da applicare al robot.

- **Controllo d'ammettenza (Causalità Forza $\rightarrow$ Moto):** misura o sintetizza una forza esterna come ingresso e la converte in un riferimento di moto (posizione, velocità e accelerazione).

Poiché i controllori industriali standard e le API di basso livello del robot sono tipicamente ottimizzati con anelli interni di posizione e velocità ad elevata rigidità, il controllo d'ammettenza risulta l'approccio ideale. Esso agisce come un filtro dinamico esterno che genera un nuovo riferimento cinematico inviabile direttamente ai servocomandi di giunto del manipolatore.

Sintesi della forza repulsiva e dinamica dell'ammettenza

Nel nostro scenario di evasione non si ha un contatto fisico reale con l'operatore; la vicinanza dell'uomo all'interno della capsula *soft* viene tradotta in una forza repulsiva virtuale $\mathbf{F}_{\text{rep}} \in \mathbb{R}^6$. Tale forza ha modulo proporzionale alla profondità di penetrazione all'interno della fascia *soft* ed è diretta lungo la retta di minima distanza tra i segmenti geometrici.

La forza virtuale $\mathbf{F}_{\text{rep}}$ viene fornita in ingresso al modello dell'ammettenza cartesiana, che modella l'interfaccia di contatto come un sistema meccanico equivalente di tipo massa-molla-smorzatore:

$$\mathbf{M}_d(\ddot{\mathbf{x}}_r - \ddot{\mathbf{x}}_d) + \mathbf{D}_d(\dot{\mathbf{x}}_r - \dot{\mathbf{x}}_d) + \mathbf{K}_d(\mathbf{x}_r - \mathbf{x}_d) = \mathbf{F}_{\text{rep}} \quad (2.21)$$

dove:

- $\mathbf{x}_d, \dot{\mathbf{x}}_d, \ddot{\mathbf{x}}_d \in \mathbb{R}^6$ indicano le grandezze cartesiane desiderate (posizione, velocità, accelerazione) pianificate dal task nominale. Lo spazio $\mathbb{R}^6$ viene adottato per descrivere completamente i 6 gradi di libertà del manipolatore nello spazio operativo, includendo sia le 3 componenti traslazionali sia le 3 componenti rotazionali dell'organo terminale;
- $\mathbf{x}_r, \dot{\mathbf{x}}_r, \ddot{\mathbf{x}}_r \in \mathbb{R}^6$ rappresentano lo stato di riferimento modificato che viene calcolato online;
- $\mathbf{M}_d, \mathbf{D}_d, \mathbf{K}_d \in \mathbb{R}^{6 \times 6}$ costituiscono le matrici simmetriche e definite positive di massa, smorzamento e rigidità virtuali desiderate.

Calcolo online della posizione modificata

L'equazione (2.21) descrive un sistema differenziale del secondo ordine. Esplicitando il termine di accelerazione modificata $\ddot{\mathbf{x}}_r$, si ottiene:

$$\ddot{\mathbf{x}}_r = \ddot{\mathbf{x}}_d + \mathbf{M}_d^{-1} \left(\mathbf{F}_{\text{rep}} - \mathbf{D}_d(\dot{\mathbf{x}}_r - \dot{\mathbf{x}}_d) - \mathbf{K}_d(\mathbf{x}_r - \mathbf{x}_d) \right) \quad (2.22)$$

Attraverso schemi di integrazione numerica in tempo reale (es. Eulero o Runge-Kutta al ciclo di controllo a 1 kHz), questa accelerazione viene integrata passo-passo per ricavare la velocità $\dot{\mathbf{x}}_r(t)$ e infine la nuova posizione cartesiana di riferimento $\mathbf{x}_r(t)$.

Il comportamento del robot durante l'evasione è governato dai parametri della matrice virtuale:

- Azione della massa ($\mathbf{M}_d$): filtra i picchi improvvisi della forza repulsiva, garantendo accelerazioni fluide ed evitando scatti bruschi.
- Azione dello smorzamento ($\mathbf{D}_d$): dissipa l'energia del sistema virtuale e previene l'insorgere di oscillazioni indesiderate durante l'allontanamento dall'operatore.
- Azione della rigidità ($\mathbf{K}_d$): agisce come un richiamo elastico verso la traiettoria nominale $\mathbf{x}_d(t)$. Nel momento in cui l'operatore esce dalla capsula *soft*, la forza repulsiva $\mathbf{F}_{\text{rep}}$ si annulla e il termine K_d riporta gradualmente e in modo fluido il robot sul percorso pianificato originario.

Infine, la posizione cartesiana modificata $\mathbf{x}_r(t)$ viene convertita nelle corrispondenti variabili di giunto $\mathbf{q}_r(t)$ tramite cinematica inversa differenziale (sfruttando lo Jacobiano analitico o geometrico del robot) e inviata all'anello di controllo di posizione interno.

Capitolo 3

Implementazione

3.1 Braccio robotico

Il manipolatore utilizzato in questa attività di tesi è il *Franka Emika Panda* mostrato in Figura 3.1, un robot collaborativo a 7 gradi di libertà costruito appositamente per l'interazione sicura con l'operatore umano.



Figura 3.1: Robot collaborativo *Franka Emika Panda*

Il robot ha una lunghezza massima di 850 mm e un carico utile *payload* nominale di 3 kg. La struttura a 7 gradi di libertà introduce una ridondanza

cinematica che consente di evitare singolarità e ostacoli nello spazio di lavoro, mantenendo inalterata la posa dell'end-effector.

Alcune applicazioni comuni includono assemblaggio di precisione, operazioni di *pick-and-place* e ispezione di qualità. L'elemento distintivo del Franka Emika Panda è l'integrazione di sensori di coppia ai giunti (*joint torque sensors*) su ciascuno dei 7 assi. Questi consentono:

- **Controllo in coppia e impedenza:** Possibilità di regolare la rigidità del robot per interagire in modo conforme con l'ambiente circostante;
- **Sicurezza intrinseca:** Rilevamento tempestivo di collisioni non pianificate con l'operatore umano o la cella di lavoro, garantendo l'arresto immediato o reazioni elastiche;
- **Apprendimento per dimostrazione:** Programmazione intuitiva tramite guida manuale (*lead-through programming*).

3.2 Architettura software e strumenti di modellazione

3.2.1 Controllo del robot e libreria *libfranka*

Per l'interfaccia a basso livello con il manipolatore è stata impiegata la libreria *libfranka*, una libreria nativa in C++ fornita da Franka Emika che si connette direttamente alla *Franka Control Interface* (FCI) tramite connessione di rete Ethernet dedicata su protocollo UDP/IP.

La FCI consente di accedere allo stato completo del robot e di inviare comandi di controllo a una frequenza di 1 kHz (1 ms di tempo di ciclo). Per garantire l'esecuzione in tempo reale e prevenire fenomeni di *jitter* o perdita di pacchetti (che causerebbero l'arresto di sicurezza del robot da parte dell'unità di controllo interna), il sistema operativo dell'elaboratore di controllo è munito di un kernel Linux con patch di tempo reale *PREEMPT_RT*.

Il paradigma di controllo di *libfranka* si basa su un ciclo di esecuzione bloccante gestito tramite la chiamata al metodo `robot.control()`, il quale accetta una funzione *lambda* (o un oggetto funzione) eseguita ad ogni passo di integrazione di 1 ms [10]. La funzione riceve in ingresso la struttura `franka::RobotState`, contenente lo stato istantaneo del robot (posizioni e velocità giuntali, accelerazioni, stime delle forze esterne, matrici jacobiane e vettori di gravità/Coriolis), e restituisce le variabili di comando desiderate.

In base alla natura delle variabili scambiate, la libreria consente di implementare diversi schemi di controllo:

- **Controllo in posizione/velocità nello spazio dei giunti:** il codice utente definisce direttamente i valori desiderati per i giunti q o le loro derivate $\dot{q}$. Il controllore interno della FCI gestisce la dinamica e calcola le coppie ai giunti necessarie per inseguire il profilo fornito.
- **Controllo in posizione/velocità nello spazio cartesiano:** l'utente specifica la posa desiderata dell'end-effector T_{EE} o il twist cartesiano v_{EE} . L'unità di controllo interna si occupa della risoluzione della cinematica inversa e del calcolo delle azioni di controllo.
- **Controllo di coppia giuntale:** in questa modalità, il controllo interno di posizione viene disabilitato e il programma applica un vettore di coppie τ_d direttamente ai giunti del robot. Questa modalità è indispensabile per l'implementazione di algoritmi avanzati sviluppati a livello utente, quali controllori a impedenza, schemi di compensazione della gravità o algoritmi basati sulla dinamica inversa del manipolatore.

Inoltre, *libfranka* offre la possibilità di combinare simultaneamente un *Motion Generator* (in posizione o velocità) e un *Controller* di coppia. Questa struttura modulare permette, ad esempio, di sovrapporre un'azione di regolazione di forza ad una traiettoria cartesiana generata in tempo reale.

Gestione della sicurezza e delle eccezioni

Trattandosi di un manipolatore collaborativo operante a frequenze elevate, la gestione della sicurezza è integrata nativamente nell'architettura software di *libfranka*. Durante l'esecuzione del loop di controllo a 1 kHz, la FCI esegue una verifica continua sul rispetto dei vincoli fisici del robot. Se vengono superati i limiti operativi — quali velocità, accelerazioni, coppie ai giunti massime o disallineamenti temporali nei pacchetti UDP — l'unità di controllo interrompe immediatamente il moto sollevando un'eccezione di tipo `franka::ControlException`.

Per garantire la robustezza del codice e la protezione dell'hardware, le chiamate ai metodi di controllo vengono tipicamente racchiuse all'interno di blocchi `try-catch`:

```

1  try {
2      robot.control(torque_control_callback);
3  } catch (const franka::ControlException& e) {
4      std::cerr << "Errore di controllo rilevato: " << e.
5          what() << std::endl;
6  }
```

Inoltre, prima di avviare qualsiasi ciclo di controllo, è buona norma impostare esplicitamente i limiti di sicurezza software tramite i metodi dedicati della classe `franka::Robot`:

- **setCollisionBehavior**: definisce le soglie di coppia e di forza cartesiana per il rilevamento delle collisioni e dei contatti sia in fase di stazionamento sia durante il moto;
- **setJointImpedance** / **setCartesianImpedance**: configurano la rigidità interna del controllore di base del robot;
- **setLimits**: impone limiti software restrittivi su velocità e accelerazioni giuntali per prevenire traiettorie troppo aggressive durante la fase di test degli algoritmi.

Questo doppio livello di protezione garantisce che qualsiasi anomalia nel codice utente si traduca in un arresto sicuro del braccio senza rischi per l'operatore o la struttura meccanica.

3.2.2 Gestione della dinamica: *Pinocchio*

Per la valutazione efficiente della cinematica e della dinamica del braccio robotico è stata impiegata la libreria open-source C++ *Pinocchio* [11]. Sviluppata prevalentemente per applicazioni di controllo e analisi di sistemi multi-corpo in tempo reale, *Pinocchio* basa la propria algebra lineare sulla libreria *Eigen* ed implementa algoritmi ricorsivi di dinamica rigida basati sull'algebra spaziale (*Spatial Algebra*) [11].

Architettura software e separazione Model/Data

A differenza di altri moduli software di dinamica, *Pinocchio* introduce una rigida separazione tra le strutture dati che descrivono il sistema e i buffer di calcolo usati dagli algoritmi:

- **Classe Model**: memorizza i parametri strutturali costanti del robot (catena cinematica, masse, inerzie e giunti) letti tipicamente da file di descrizione come URDF;
- **Classe Data**: contiene i risultati intermedi, i buffer di memoria temporanei e le grandezze risultanti (posizioni, velocità, accelerazioni e matrici jacobiane).

Questa separazione evita allocazioni dinamiche di memoria durante l'esecuzione e garantisce una firma uniforme per tutti gli algoritmi nella forma `algorithm(model, data, q, v, ...)`, favorendo l'esecuzione parallela e la prevedibilità dei tempi di calcolo.

La correttezza dei calcoli dinamici svolti da *Pinocchio* dipende in maniera critica dalla fedeltà del modello URDF fornito in ingresso. Poiché i file URDF modellano la struttura come una catena di corpi rigidi con giunti ideali, gli attriti interni e altre dinamiche non modellate vengono tralasciati (aspetto che verrà affrontato nel dettaglio in Sezione 3.2.3).

Efficienza computazionale via polimorfismo statico

Per coniugare la flessibilità del caricamento dinamico del modello con l'efficienza tipica del codice generato **ad hoc**, *Pinocchio* adotta il paradigma del polimorfismo statico basato sul design pattern CRTP (*Curiously Recurring Template Pattern*).

Mentre il polimorfismo dinamico tradizionale (basato su funzioni virtuali) introduce rallentamenti dovuti alla risoluzione a tempo di esecuzione e ostacola le ottimizzazioni del compilatore, il meccanismo CRTP seleziona gli operatori specifici di ciascun giunto direttamente in fase di compilazione. Combinando il pattern CRTP con l'uso dei *variant* per scorrere i giunti del modello, la libreria minimizza l'overhead di memoria e ottimizza l'uso delle risorse di calcolo dell'elaboratore. Questo rende la libreria ideale sia per il ciclo di controllo in tempo reale a 1 kHz, sia per l'integrazione all'interno di routine di ottimizzazione numerica.

Algoritmi dinamici e derivate analitiche

All'interno del loop di controllo a basso livello, *Pinocchio* consente di eseguire in tempi dell'ordine del microsecondo i principali algoritmi della dinamica rigida:

- **RNEA (*Recursive Newton-Euler Algorithm*)**: calcola la dinamica inversa per determinare le coppie ai giunti τ necessarie a realizzare il moto desiderato;
- **CRBA (*Composite Rigid Body Algorithm*)**: calcola la matrice di inerzia nello spazio dei giunti $M(q)$;
- **ABA (*Articulated Body Algorithm*)**: risolve la dinamica diretta ottenendo le accelerazioni dei giunti a partire dalle coppie applicate.

Un elemento caratterizzante di *Pinocchio* è la presenza nativa delle derivate analitiche di tali algoritmi. Ciò ne permette l'impiego diretto all'interno di schemi di controllo in spazio di stato, stimatori basati su filtro di Kalman o algoritmi di controllo ottimo e *Model Predictive Control* (MPC).

3.2.3 Modellazione attriti ai giunti

Per valutare l'accuratezza del modello cinematico del robot implementato in C++ tramite la libreria Pinocchio, è stato effettuato un confronto tra le coppie reali misurate ai giunti e quelle stimate dal modello URDF.

Inizialmente, considerando esclusivamente i parametri inerziali presi dal modello CAD dell'URDF, si nota che la curva della coppia stimata ricalca l'andamento di quella reale, ma presenta un offset significativo. Tale discrepanza è attribuibile alla mancata modellazione dei fenomeni dissipativi, come l'attrito viscoso e l'attrito di Coulomb presenti nei riduttori e nei cuscinetti dei giunti [12].

Per modellare accuratamente tali effetti dissipativi e garantire la continuità del modello nelle simulazioni dinamiche, è stata integrata una formulazione per la coppia d'attrito $\tau_{f,j}$ al giunto j . Nella sua forma classica l'attrito è rappresentato dalla combinazione di attrito viscoso e di Coulomb:

$$\tau_{f,j}(\dot{q}_j) = f_{v,j}\dot{q}_j + f_{c,j}\operatorname{sgn}\dot{q}_j + f_{o,j} \quad (3.1)$$

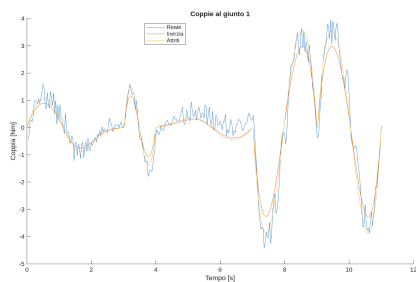
dove $f_{v,j}$ rappresenta il coefficiente di attrito viscoso, $f_{c,j}$ l'attrito di Coulomb e $f_{o,j}$ un eventuale offset [12].

Tuttavia, per evitare le discontinuità e i fenomeni di *chattering* in prossimità dello zero ($\dot{q}_j \approx 0$), si fa uso di una formulazione sigmoideale continua [12]:

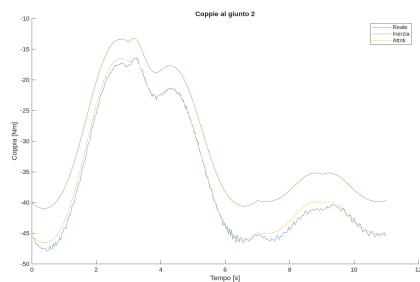
$$\tau_{f,j}(\dot{q}_j) = c_{v,j}\dot{q}_j + \frac{\varphi_{1,j}}{1 + e^{-\varphi_{2,j}(\dot{q}_j + \varphi_{3,j})}} - \frac{\varphi_{1,j}}{1 + e^{-\varphi_{2,j}\varphi_{3,j}}} \quad (3.2)$$

dove i parametri $\varphi_{1,j}, \varphi_{2,j}, \varphi_{3,j}$ (unitamente agli eventuali termini viscosi) identificano la risposta dinamica non lineare del singolo giunto [12].

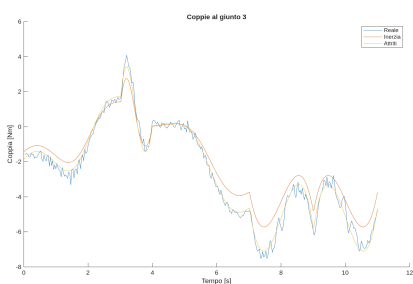
L'introduzione nel modello di Pinocchio dei parametri di smorzamento (*damping*) e attrito (*friction*) permette di eliminare l'offset residuo. La curva di coppia calcolata dal modello identificato risulta così sovrapponibile a quella misurata dai sensori di giunto, Figura 3.2, confermando l'importanza dell'inclusione dei parametri dissipativi per una stima dinamica accurata sia tramite approccio lagrangiano sia nell'algoritmo ricorsivo di Newton-Euler [12].



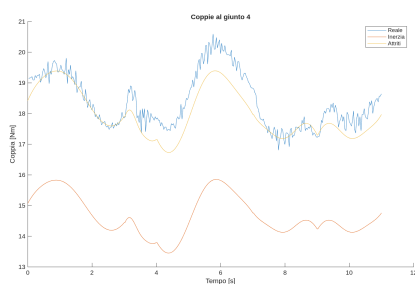
(a) Giunto 1



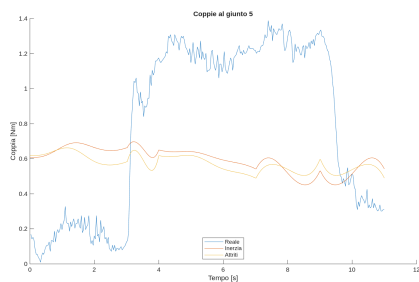
(b) Giunto 2



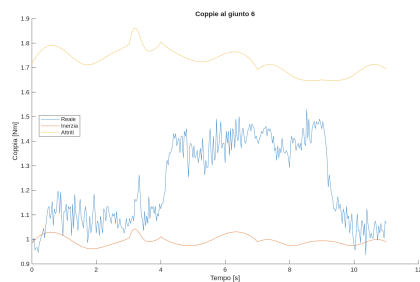
(c) Giunto 3



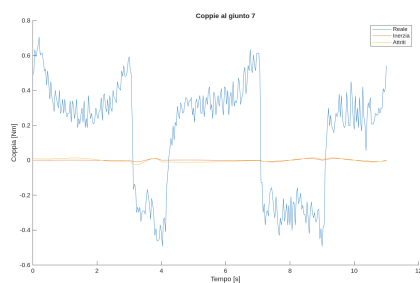
(d) Giunto 4



(e) Giunto 5



(f) Giunto 6



(g) Giunto 7

Figura 3.2: Confronto tra le coppie reali misurate ai giunti e le coppie stimate tramite il modello dinamico in Pinocchio.

3.2.4 Ambiente di simulazione

Per consentire una rapida validazione del codice ed evitare rischi legati a malfunzionamenti dell'hardware reale, è stata sviluppata una libreria di simulazione in C++ che rispecchia l'interfaccia di `libfranka`.

In particolare il metodo `robot.control(...)` è stato implementato adottando un modello di controllo ideale, in cui il robot insegue i comandi ricevuti in modo perfetto. Il Listato 3.1 mostra l'implementazione del loop di controllo principale a 1kHz.

Listing 3.1: Implementazione del controllo ideale nell'ambiente di simulazione.

```
1  // Mirror robot.control(), runs callback at 1kHz
2  void control(std::function<JointPositions(const
    RobotState&, Duration)> callback) {
3      Duration period;
4
5      while (true) {
6          JointPositions cmd = callback(state_, period);
7
8          // Update simulated state, assume robot perfectly
           tracks commands
9          for (int j = 0; j < 7; ++j) {
10             state_.dq[j] = (cmd.q[j] - state_.q[j]) / period.
                toSec();
11             state_.ddq_d[j] = (state_.dq[j] - prev_dq_[j]) /
                period.toSec();
12             prev_dq_[j] = state_.dq[j];
13             state_.q[j] = cmd.q[j];
14             state_.q_d[j] = cmd.q_d[j];
15         }
16
17         if (cmd.motion_finished) break;
18
19         std::this_thread::sleep_for(std::chrono::
                milliseconds(1));
20     }
21 }
```

L'architettura della libreria si ispira al lavoro di Oliva et al. [13]. Trattandosi di un simulatore cinematico privo di un modello dinamico o fisico dettagliato del manipolatore, l'ambiente virtuale non contempla gli attriti, i

goichi meccanici e le non idealità proprie del robot reale (trattate in dettaglio nella Sezione 3.4.3).

3.3 Sistema di visione e playback dello scheletro

L'acquisizione visiva della scena viene effettuata tramite una telecamera di profondità Intel Realsense D435. Il dispositivo calcola la mappa di profondità direttamente a bordo tramite l'ASIC dedicato, sgravando l'unità di calcolo ospite da tale carico computazionale [14].

A partire dalle immagini acquisite, la posa dello scheletro umano viene stimata mediante l'architettura a rete neurale YOLOv8 [7, 15], i punti salienti (*keypoints*) estratti vengono successivamente salvati su file di testo.

La fase di playback dello scheletro registrato si avvale della libreria ZeroMQ [16], strutturata secondo un paradigma di comunicazione monodirezionale di tipo *Publisher-Subscriber*. Per garantire il rispetto dei vincoli temporali del ciclo di controllo a 1kHz, l'architettura software impiega due thread distinti:

- **Nodo Python (*Publisher*):** trasmette le coordinate dei punti salienti dello scheletro stimato;
- **Nodo C++ (*Subscriber*):** riceve i dati in streaming e li memorizza in un vettore condiviso (*Skeleton*) accessibile dal programma di controllo principale.

3.4 Primo metodo: frenata morbida

Come visto in Sezione 2.3, è necessario risolvere un problema di ottimizzazione vincolata. Per fare ciò si è deciso di utilizzare il framework open-source *CasADi* [17].

3.4.1 Ottimizzazione tramite CasADi

CasADi è uno strumento software ad alte prestazioni sviluppato per l'ottimizzazione numerica, con particolare attenzione all'ottimizzazione vincolata non lineare (NLP) e al controllo ottimale. È scritto in C++ con interfacce per Python e MATLAB, rendendolo estremamente versatile sia in fase di prototipazione che di implementazione in sistemi embedded o ad alte prestazioni.

Il punto di forza di CasADi risiede nella sua struttura simbolica basata sui grafi di espressione (*expression graphs*). Ciò consente di eseguire la Differenziazione Algoritmica (AD) sia in modalità diretta (*forward mode*) sia inversa (*reverse mode*). In questo modo, derivate prime (gradienti, Jacobiane) e derivate seconde (Hessiane) vengono calcolate con accuratezza pari alla precisione della macchina, evitando le imprecisioni numeriche della differenziazione alle differenze finite e l'esplosione della complessità del calcolo simbolico classico.

Un generico problema di ottimizzazione non lineare (NLP) risolto da CasADi viene formulato nella seguente forma standard:

$$\min_x f(x) \text{ s.t. } x_{lb} \leq x \leq x_{ub} \quad g_{lb} \leq g(x) \leq g_{ub} \quad (3.3)$$

dove $x \in \mathbb{R}^n$ rappresenta il vettore delle variabili decisionali, $f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$ è la funzione obiettivo (cost function), x_{lb} e x_{ub} sono i limiti inferiori e superiori sulle variabili (box constraints), mentre $g(x) : \mathbb{R}^n \rightarrow \mathbb{R}^m$ rappresenta il vettore delle funzioni di vincolo non lineare bounded da g_{lb} e g_{ub} .

Per la risoluzione del problema, CasADi mette a disposizione l'interfaccia ad alto livello `casadi::nlpsol`. Questa interfaccia accetta un dizionario `nlp` contenente la definizione simbolica di x , $f(x)$ e $g(x)$, unita alla specifica del solutore numerico sottostante (come ad esempio *IPOPT*, *SNOPT* o *KNITRO*).

Esempio di risoluzione NLP in C++

Di seguito viene mostrato un esempio applicativo in C++ per la risoluzione di un problema di ottimizzazione non lineare limitato e vincolato Eq. 3.4 tramite il solutore IPOPT integrato in CasADi:

$$\min_{x_0, x_1} x_0^2 + (x_1 - 1)^2 \quad \text{s.t.} \quad x_0 + x_1 = 2, \quad x_0 \geq 0, ; x_1 \geq 0 \quad (3.4)$$

```

1 #include <casadi/casadi.hpp>
2 #include
3
4 using namespace casadi;
5
6 int main() {
7 // 1. Definizione delle variabili simboliche (SX o MX)
8 SX x = SX::sym("x", 2); // Vettore di 2 variabili
   decisionali (x0, x1)
9 // 2. Definizione della funzione costo f(x) e dei vincoli
   g(x)

```

```

10 SX f = pow(x(0), 2) + pow(x(1) - 1, 2);
11 SX g = x(0) + x(1);
12
13 // 3. Creazione del dizionario NLP per CasADi
14 MXDict nlp = {
15 {"x", x},
16 {"f", f},
17 {"g", g}
18 };
19
20 // 4. Inizializzazione del solutore (es. IPOPT)
21 // E possibile passare opzioni aggiuntive come secondo
    argomento (Dict)
22 Function solver = nlpsol("solver", "ipopt", nlp);
23
24 // 5. Impostazione dei limiti numerici sui vincoli e
    sulle variabili
25 DNDict args;
26 args["x0"] = std::vector<double>{0.0, 0.0}; // Punto di
    partenza (Initial guess)
27
28 // Bounds sulle variabili x (x >= 0)
29 args["lbx"] = std::vector<double>{0.0, 0.0};
30 args["ubx"] = std::vector<double>{inf, inf};
31
32 // Bounds sul vincolo g(x) = 2 -> lbg = ubg = 2
33 args["lbg"] = std::vector<double>{2.0};
34 args["ubg"] = std::vector<double>{2.0};
35
36 // 6. Risoluzione del problema
37 DNDict res = solver(args);
38
39 // 7. Estrazione dei risultati
40 std::vector<double> x_opt = std::vector<double>(res["x"])
    ;
41 std::cout << "Soluzione ottimale x: [" << x_opt[0] << ",
    " << x_opt[1] << "]" << std::endl;
42 std::cout << "Valore minimo costo: " << double(res["f"])
    << std::endl;
43
44 return 0;
45
46 }

```

3.4.2 Rilevamento di una potenziale collisione

La libreria *CasADi* descritta nella sezione precedente viene utilizzata per risolvere il problema di ottimizzazione definito nella Sezione 2.3.3, vincolato ai limiti di posizione, velocità, accelerazione e coppia dei giunti del robot:

$$\min_{T_{\text{stop}}} w_0 T_{\text{stop}} + w_1 |T_{\text{stop}} - T_{\text{stop,prev}}| \quad (3.5)$$

Per garantire il rispetto dei vincoli temporali e non bloccare l'esecuzione del task nominale, l'architettura software sfrutta due thread dedicati:

- **skeleton_thread**: gestisce la ricezione e il playback in tempo reale delle posizioni delle articolazioni dello scheletro umano;
- **collision_checker_thread**: esegue la risoluzione dell'ottimizzazione in Eq. (3.5) nell'intervallo $T_{\text{stop}} \in [0.1, 0.4]$ s a partire dallo stato corrente del robot letto tramite *libfranka*, integrando il modello cinematico e dinamico fornito da *Pinocchio*.

I parametri geometrici, cinematici e numerici adottati nell'implementazione dell'algoritmo sono sintetizzati nella Tabella 3.1.

Per garantire un approccio cautelativo (worst-case scenario), la velocità massima dell'umano $v_{\text{human, max}}$ è stata impostata a 1.60 m/s, in accordo con le direttive sulle velocità di avvicinamento stabilite dalla norma ISO/TS 15066 per applicazioni collaborative. I raggi delle primitive SSL (r_r e r_h) coprono l'ingombro effettivo dei link del robot Franka Emika Panda e dei segmenti corporei dell'operatore.

A partire dal tempo T_{stop} ottimizzato ad ogni ciclo, la distanza di separazione minima di sicurezza d_{safe} considerata nell'algoritmo viene calcolata dinamicamente come:

$$d_{\text{safe}} = v_{\text{human, max}} \cdot (T_{\text{reaction}} + T_{\text{stop}}) \quad (3.6)$$

Se la distanza tra i volumi SSL del robot e dell'umano risulta inferiore a d_{safe} , viene rilevata una potenziale collisione e attivata la traiettoria di arresto entro il tempo T_{stop} .

3.4.3 Esecuzione del Task

L'applicazione di test sviluppata per la validazione delle strategie di sicurezza nella collaborazione uomo-robot (HRC) consiste in un task sequenziale di *pick and place*. Dal punto di vista software, la logica di controllo in C++ è stata architettata ed eseguita come una macchina a stati finiti (FSM, *Finite State Machine*), garantendo una gestione deterministica delle transizioni tra la traiettoria nominale e le manovre di arresto di sicurezza.

Tabella 3.1: Parametri geometrici, cinematici e di configurazione usati dall'algoritmo di rilevamento collisione.

Parametro	Simbolo	Valore	Unità di misura
<i>Modello geometrico (Capsule SSL)</i>			
Raggio capsula 1 robot	$r_{r,1}$	0.085	m
Raggio capsula 2 robot	$r_{r,2}$	0.085	m
Raggio capsula 3 robot	$r_{r,3}$	0.060	m
Raggio capsula 4 robot	$r_{r,4}$	0.065	m
Raggio capsule scheletro umano	r_h	0.05 – 0.16	m
<i>Parametri cinematici e tempi</i>			
Velocità massima stimata umano	$v_{\text{human, max}}$	1.60	m/s
Tempo di reazione del sistema	T_{reaction}	0.05	s
<i>Vincoli e impostazioni ottimizzatore (CasADi)</i>			
Tempo di stop minimo	$T_{\text{stop, min}}$	0.10	s
Tempo di stop massimo	$T_{\text{stop, max}}$	0.40	s
Peso termine tempo di stop	w_0	10^6	–
Peso variazione tempo di stop	w_1	10^6	–
Numero massimo di iterazioni	<code>max_iter</code>	5	–

Architettura Software e Flusso di Esecuzione

A livello concettuale, la macchina a stati si fonda su due macro-stati operativi principali:

- **RUNNING**: il manipolatore insegue la traiettoria nominale pianificata nello spazio dei giunti o cartesiano. Al raggiungimento di ciascun *way-point* intermedio, l'algoritmo provvede al calcolo e all'interpolazione del segmento di traiettoria successivo.
- **STOPPING**: innescato dal rilevamento di una condizione di potenziale pericolo o dalla violazione delle distanze di sicurezza, questo stato forza l'esecuzione di una traiettoria di arresto a tempo ottimo (*time-optimal stopping trajectory*). Al termine della decelerazione, il robot viene mantenuto in uno stato di arresto temporaneo per un intervallo di tempo prefissato, durante il quale il sistema calcola la nuova traiettoria di ripartenza in modo da garantire un rientro fluido e privo di urti (*jerk-free*) sulla traiettoria nominale.

Dal punto di vista dell'implementazione software, la FSM non è strutturata all'interno di un unico blocco condizionale, ma si articola attraverso una gerarchia di funzioni che separano la *pianificazione di alto livello* dal *loop di controllo ad alta frequenza* (1kHz). Il flusso operativo segue una logica ciclica suddivisa nei seguenti passi:

1. **Pianificazione e Inizializzazione (Alto Livello)**: La funzione principale di gestione (*executeTask*) configura i dati di input, calcola analiticamente i segmenti della traiettoria desiderata e invoca l'esecutore di movimento ad alta frequenza.
2. **Esecuzione Real-Time (1kHz)**: La funzione di controllo a 1kHz (*executeTrajectory*, implementata tramite costrutto *lambda* in C++) esegue l'inseguimento della traiettoria generata. Il loop controlla continuamente le condizioni di terminazione o di interruzione:
 - Se il robot raggiunge la configurazione finale nominale, la funzione solleva la condizione di fine movimento (`franka::MotionFinished`) e restituisce il controllo al livello superiore.
 - Se si verifica una condizione di stop per sicurezza, il sistema commuta nello stato **STOPPING**, porta il robot all'arresto in tempo ottimo e termina l'esecuzione del ciclo ad alta frequenza.

3. **Fase di Stasi e Ri-pianificazione (*Re-planning State*):** Al rientro dal loop real-time, il programma entra in uno stato transitorio di stasi. In questo intervallo il robot rimane fermo in posizione di sicurezza, mentre il thread di alto livello valuta la nuova configurazione e ricalcola la traiettoria di rientro prima di richiamare nuovamente il ciclo di controllo a 1 kHz.

La Figura ?? illustra lo schema concettuale di tale architettura. Come si può notare, il sistema commuta in modo deterministico tra il livello di pianificazione ad alto livello e il loop di controllo ad alta frequenza. Quando la traiettoria viene completata o interrotta nello stato **STOPPING**, la restituzione dell'oggetto `franka::MotionFinished` riporta l'esecuzione allo stato di stasi ad alto livello, consentendo il ricalcolo sicuro della nuova traiettoria senza saturare il thread real-time.

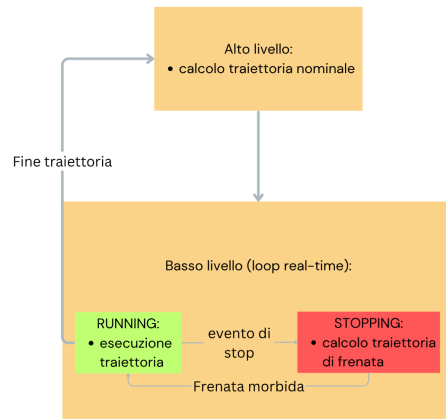


Figura 3.3: Architettura software del sistema di controllo. L'alto livello gestisce la pianificazione offline e la ri-pianificazione delle traiettorie, mentre il basso livello esegue il loop di controllo ad alta frequenza (1 kHz) alternando deterministicamente gli stati di **RUNNING** e **STOPPING**.

Nel seguito vengono analizzate ed elaborate due strategie di controllo implementate tramite le API di basso livello di `libfranka`: il controllo in posizione e il controllo in coppia.

Controllo di Posizione dei Giunti

In prima istanza, la movimentazione del robot è stata implementata sfruttando l'interfaccia di controllo in posizione fornita da `libfranka`, mediante il metodo generico:

```

1 robot.control([&](const franka::RobotState& state, franka
  ::Duration period)
2 -> franka::JointPositions { ... }));

```

Attraverso questo approccio, la funzione *lambda* viene eseguita all'interno del loop di controllo ad alta frequenza (1 kHz) e restituisce al controllore interno del manipolatore i valori obiettivo per le posizioni dei giunti ($\mathbf{q}_d$).

Sebbene questo approccio abbia mostrato prestazioni soddisfacenti nell'ambiente di simulazione, l'applicazione sul robot reale ha evidenziato criticità applicative. In particolare, si sono riscontrate discontinuità nel profilo delle velocità misurate ai giunti, accompagnate da sollecitazioni meccaniche udibili e vibrazioni indesiderate sulla struttura del manipolatore. Tale fenomeno è riconducibile al fatto che l'invio diretto di posizioni discrete, se non accuratamente filtrato a livello di *jerk*, può generare derivate prime e seconde non continue, attivando i filtri di protezione interni del robot o causandone un comportamento a scatti.

Per la gestione dell'arresto definitivo o del completamento del movimento, l'uscita dal ciclo di controllo viene notificata al sistema restituendo l'oggetto di terminazione con lo stato finale desiderato, mantenendo la posizione corrente di riferimento:

```

1 return franka::MotionFinished(franka::JointPositions(
  state.q_d));

```

Controllo di Coppia e Controllo di Impedenza

Per risolvere le discontinuità nei profili di velocità e garantire un'interazione intrinsecamente sicura durante l'esecuzione del task, si è scelto di implementare uno schema di controllo di coppia (*torque control*) in C++. L'interfaccia sfrutta il seguente metodo:

```

1 robot.control([&](const franka::RobotState& state, franka
  ::Duration period)
2 -> franka::Torques { ... }));

```

Per garantire un inseguimento preciso e reattivo della traiettoria, è stata adottata una legge di controllo a coppia calcolata (*Computed Torque Control*), che unisce un'azione di *feedforward* (basata sul modello dinamico inverso del robot) ad un'azione di *feedback* proporzionale-derivativa (PD) per regolare l'impedenza ai giunti.

Matematicamente, la coppia desiderata $\tau_d \in \mathbb{R}^7$ da applicare ai giunti è calcolata come:

$$\tau_d = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}}_d + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{K}_p(\mathbf{q}_d - \mathbf{q}) + \mathbf{K}_d(\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) \quad (3.7)$$

dove:

- $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{7 \times 7}$ è la matrice di inerzia dipendente dalla configurazione attuale $\mathbf{q}$;
- $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^7$ è il vettore che raggruppa gli effetti di Coriolis e le forze centrifughe;
- $\mathbf{q}_d, \dot{\mathbf{q}}_d, \ddot{\mathbf{q}}_d \in \mathbb{R}^7$ sono rispettivamente posizione, velocità ed accelerazione desiderate ai giunti, calcolate dal generatore di traiettoria;
- $\mathbf{K}_p, \mathbf{K}_d \in \mathbb{R}^{7 \times 7}$ sono le matrici diagonali dei guadagni (rispettivamente di rigidità e smorzamento) del controllore di impedenza.

*Nota: Il vettore della gravità $\mathbf{g}(\mathbf{q})$ non compare esplicitamente nella legge di controllo, poiché l'interfaccia **libfranka** provvede, di default, a sommare la compensazione di gravità interna ai comandi di coppia forniti dall'utente.*

A livello software, l'implementazione nel loop di controllo a 1 kHz necessita della classe `franka::Model` per recuperare i parametri dinamici aggiornati. Poiché le API di **libfranka** restituiscono questi dati sotto forma di `std::array`, si è fatto ricorso alla libreria **Eigen** e in particolare alla classe `Eigen::Map`, che permette di proiettare gli array in matrici e vettori senza allocare nuova memoria, ottimizzando così i tempi di calcolo *real-time*:

```

1 // Acquisizione dei termini dinamici dal modello di
   libfranka
2 std::array<double, 49> mass_array = model.mass(state);
3 std::array<double, 7> coriolis_array = model.coriolis(
   state);
4
5 // Mappatura (zero-copy) degli array in strutture dati
   Eigen
6 Eigen::Map<Eigen::Matrix<double, 7, 7>> M(mass_array.data
   ());
7 Eigen::Map<Eigen::Matrix<double, 7, 1>> C(coriolis_array.
   data());
8 Eigen::Map<Eigen::Matrix<double, 7, 1>> q(state.q.data())
   ;
9 Eigen::Map<Eigen::Matrix<double, 7, 1>> dq(state.dq.data
   ());

```

```

10
11 // Calcolo degli errori di inseguimento (feedback)
12 Eigen::Matrix<double, 7, 1> error = q_d - q;
13 Eigen::Matrix<double, 7, 1> derror = dq_d - dq;
14
15 // Legge di controllo a coppia calcolata
16 Eigen::Matrix<double, 7, 1> tau_d;
17 tau_d = M * ddq_d + C + K_p * error + K_d * derror;
18
19 // Invio del comando al manipolatore
20 std::array<double, 7> tau_d_array;
21 Eigen::VectorXd::Map(&tau_d_array[0], 7) = tau_d;
22 return tau_d_array;

```

L'adozione di questo schema ha permesso non solo di eliminare le discontinuità e il rumore causati dal controllo di posizione, ma anche di conformare il comportamento dinamico del robot in caso di contatto, un requisito fondamentale per le applicazioni di robotica collaborativa.

3.5 Secondo approccio: capsule *soft*

L'architettura del controllore si basa su una macchina a stati finiti (FSM) analoga a quella presentata nella sezione precedente. La differenza fondamentale risiede nello stato di *RUNNING*, che integra un modulo di verifica per le interazioni con le capsule *soft* esterne per la generazione delle corrispondenti forze repulsive.

Quando una capsula associata all'operatore entra in contatto con una capsula *soft* del robot, il sistema calcola una forza virtuale repulsiva di tipo elastico. L'entità di tale forza è proporzionale alla penetrazione tra le capsule ed è modulata mediante una funzione di raccordo continuo (*smoothstep*) [7].

La forza risultante viene convertita nella variazione di posa cartesiana dell'meccano-struttura (*end-effector*) per il passo di controllo successivo, implementando il paradigma del controllo di ammettenza esplicitato nella Sezione 2.4.2. In particolare, la dinamica dell'interazione viene modellata secondo il classico sistema massa-molla-smorzatore:

$$\mathbf{M}\ddot{\mathbf{x}}(t) + \mathbf{D}\dot{\mathbf{x}}(t) + \mathbf{K}(\mathbf{x}(t) - \mathbf{x}_0) = \mathbf{F}_{ext}(t) \quad (3.8)$$

dove M , D e K rappresentano rispettivamente le matrici di massa, smorzamento e rigidità virtuali, $x(t)$ è la posa cartesiana dell'*end-effector* e $F_{ext}(t)$ indica la forza repulsiva risultante generata dal modello a capsule.

Oltre ai volumi esterni, il modello integra internamente delle capsule *hard*, dimensionate ed ottimizzate con la stessa metodologia descritta nella Sezione 3.4. Nel caso in cui si verifichi una violazione di tali volumi interni, il controllore interrompe l'esecuzione e comanda una frenata morbida del robot nell'intervallo di tempo ottimizzato T_{stop} .

Capitolo 4

Risultati

4.1 BOZZA

metti grafici di controllo di coppia vs posizione

4.2 Controllo di posizione e di coppia ai giunti

4.3 Primo approccio

4.4 Secondo approccio

Capitolo 5

Conclusioni e sviluppi futuri

sviluppi futuri: implementazione della predizione del moto dell'operatore
(tipo con klt) per resistere a occlusioni

Bibliografia

- [1] A. Baratta, A. Cimino, M. G. Gnoni, and F. Longo. Human robot collaboration in industry 4.0: a literature review. *Procedia Computer Science*, 217:1887–1895, 2023.
- [2] E. Montini, F. Daniele, L. Agbomemewa, M. Confalonieri, V. Cutrona, A. Bettoni, P. Rocco, and A. Ferrario. Collaborative robotics: A survey from literature and practitioners perspectives. *Journal of Intelligent & Robotic Systems*, 110:117, 2024.
- [3] W. Li, Y. Hu, Y. Zhou, and D. T. Pham. Safe human–robot collaboration for industrial settings: a survey. *Journal of Intelligent Manufacturing*, 35:2235–2261, 2024.
- [4] International Organization for Standardization. ISO/TS 15066: 2016 robots and robotic devices-collaborative robots. Technical report, International Organization for Standardization, 2016.
- [5] J. A. Marvel and R. Norcross. Implementing speed and separation monitoring in collaborative robot workcells. *Robotics and Computer-Integrated Manufacturing*, 44:144–155, 2017.
- [6] L. Scalera, A. Giusti, R. Vidoni, and A. Gasparetto. Enhancing fluency and productivity in human-robot collaboration through online scaling of dynamic safety zones. *The International Journal of Advanced Manufacturing Technology*, 121:6783–6798, 2022.
- [7] D. Qyrani. Approcci di controllo avanzati per una collaborazione sicura tra uomo e robot. Master’s thesis, Università degli Studi di Udine, 2026.
- [8] E. Larsen, S. Gottschalk, M. C. Lin, and D. Manocha. Fast proximity queries with swept sphere volumes. Technical report, Technical Report TR99-018, Department of Computer Science, University of North Carolina, 1999.

- [9] L. Scalera, F. Lozer, A. Giusti, and A. Gasparetto. An experimental evaluation of robot-stopping approaches for improving fluency in collaborative robotics. *Robotica*, 42(5):1386–1402, 2024.
- [10] Sami Haddadin, Sven Parusel, Lars Johannsmeier, Saskia Golz, Simon Gabl, Florian Walch, Mohamadreza Sabaghian, Christoph Jähne, Lukas Hausperger, and Simon Haddadin. The franka emika robot: A reference platform for robotics research and education. *IEEE Robotics & Automation Magazine*, 29(2):46–64, 2022.
- [11] Justin Carpentier, Guilhem Saurel, Gabriele Buondonno, Joseph Mirabel, Florent Lamiriaux, Olivier Stasse, and Nicolas Mansard. The Pinocchio C++ library – A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. In *SII 2019 - International Symposium on System Integrations*, Paris, France, January 2019.
- [12] Claudio Gaz, Marco Cognetti, Alexander Oliva, Paolo Robuffo Giordano, and Alessandro De Luca. Dynamic identification of the franka emika panda robot with retrieval of feasible parameters using penalty-based optimization. *IEEE Robotics and Automation Letters*, 4(4):4147–4154, 2019.
- [13] Alexander Antonio Oliva, Fabien Spindler, Paolo Robuffo Giordano, and François Chaumette. Frankasim: A dynamic simulator for the franka emika robot with visual-servoing enabled capabilities. In *2022 17th International Conference on Control, Automation, Robotics and Vision (ICARCV)*, pages 868–874. IEEE, 2022.
- [14] Vladimir Tadic, Akos Odry, Istvan Kecskes, Ervin Burkus, Zoltan Kiri, and Peter Odry. Application of intel realsense cameras for depth image generation in robotics. *WSEAS Transac. Comput*, 18:2224–2872, 2019.
- [15] Dillon Reis, Jordan Kupec, Jacqueline Hong, and Ahmad Daoudi. Real-time flying object detection with yolov8. *arXiv preprint arXiv:2305.09972*, 2023.
- [16] Pieter Hintjens. *ZeroMQ: messaging for many applications*. " O'Reilly Media, Inc.", 2013.
- [17] Joel A E Andersson, Joris Gillis, Greg Horn, James B Rawlings, and Moritz Diehl. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 2018.